

Template Generator für Spring-Boot-Services

Deutsche Telekom IT

Technologien: Java, Spring Boot, Mustache, Spring Initializr, REST, Git

Entwicklung eines internen Template Generators zur automatisierten Erstellung neuer Spring-Boot-Services. Der Generator nutzte Mustache-Templates, um eine einheitliche Projektstruktur sowie bereits vorkonfigurierte, Telekom-spezifische Dependencies und Grundeinstellungen bereitzustellen.

Aufgaben und Schwerpunkte:

- Entwicklung und Weiterentwicklung des Template Generators
- Erstellung und Pflege von Mustache-Templates
- Automatisierte Generierung lauffähiger Spring-Boot-Projekte
- Integration Telekom-spezifischer Dependencies und Konfigurationen
- Standardisierung des initialen Projektaufbaus für neue Backend-Services
- Pflege und Anpassung des Generators an neue technische Anforderungen

Nutzen:

Der Generator vereinfachte die Erstellung neuer Spring-Boot-Services und stellte sicher, dass benötigte interne Abhängigkeiten und grundlegende Konfigurationen bereits beim Projektstart einheitlich vorhanden waren.

Deployment von Spring-Boot-Services in OpenShift

Deutsche Telekom IT

Technologien: Docker, Kubernetes, OpenShift, CI/CD, Git

Einrichtung und Automatisierung des Deployments von Spring-Boot-basierten Services in einer OpenShift-Umgebung. Schwerpunkt war die Bereitstellung der Anwendungen einschließlich der erforderlichen Deployment-Konfiguration und Betriebsressourcen.

Aufgaben und Schwerpunkte:

- Einrichtung und Anpassung von CI/CD-Pipelines für automatisierte Builds und Deployments
- Deployment von Spring-Boot-Services auf OpenShift
- Erstellung und Verwaltung von Secrets für sensible Konfigurationsdaten
- Einrichtung und Konfiguration von Kubernetes/OpenShift CronJobs
- Pflege und Anpassung der Deployment-Konfigurationen
- Analyse und Behebung von Problemen innerhalb des Deployment-Prozesses

Nutzen:

Durch die automatisierten Deployment-Prozesse konnten Services reproduzierbar und mit den benötigten Konfigurationen in der OpenShift-Umgebung bereitgestellt werden.

Verwaltungsprogramm für Sporttrainer**Hochschulprojekt**

Rolle: Product Owner / Entwickler

Technologien: Go, Gin, REST, Frontend-/Backend-Integration, Deployment, Git

Entwicklung einer Webanwendung zur Verwaltung von Sporttrainern. Neben meiner Rolle als Product Owner lag mein technischer Schwerpunkt auf der Integration des mit Go und Gin entwickelten Backends in die Deployment-Umgebung sowie auf dem Zusammenspiel zwischen Frontend und Backend.

Aufgaben und Schwerpunkte:

- Verantwortung für die Product-Owner-Rolle innerhalb des Projektteams
- Priorisierung und Abstimmung von Anforderungen
- Pflege und Strukturierung des Product Backlogs
- Abstimmung zwischen fachlichen Anforderungen und technischer Umsetzung
- Integration des Go-/Gin-Backends in die Deployment-Umgebung
- Konfiguration und Abstimmung der REST-Schnittstellen zwischen Frontend und Backend
- Unterstützung bei der Anbindung des Frontends an das Backend
- Analyse und Behebung von Integrationsproblemen
- Mitarbeit an der technischen Bereitstellung der Gesamtanwendung

Nutzen:

Durch die Kombination aus Product-Owner-Verantwortung und technischer Mitarbeit konnte ich sowohl die fachliche Ausrichtung des Projekts als auch die Integration der Anwendungskomponenten aktiv mitgestalten.

Migration eines Spring-Boot-Services zu Go/Fiber**Deutsche Telekom IT**

Technologien: Go, Fiber, Java, Spring Boot, Kubernetes, AWS, Docker, Git

Migration eines bestehenden Spring-Boot-Services auf eine leichtgewichtige Implementierung mit Go und dem Fiber-Framework. Zusätzlich war ich für die Bereitstellung des migrierten Services auf einem Kubernetes-Cluster in AWS zuständig.

Aufgaben und Schwerpunkte:

- Analyse des bestehenden Spring-Boot-Services
- Migration der bestehenden Funktionalität von Java/Spring Boot zu Go/Fiber
- Anpassung von Schnittstellen und Anwendungsconfiguration
- Containerisierung des Go-Services
- Erstellung und Anpassung der Kubernetes-Deployment-Konfiguration
- Deployment des Services auf einem Kubernetes-Cluster in AWS
- Analyse und Behebung von Problemen während Migration und Deployment
- Überprüfung der Funktionsfähigkeit des migrierten Services

Nutzen:

Durch die Migration konnte der bestehende Service auf eine schlankere Go-basierte Architektur übertragen und anschließend containerisiert in einer AWS-basierten Kubernetes-Umgebung betrieben werden.

Hochschulportal mit Keycloak-Integration

Hochschulprojekt

Technologien: Java, Spring Boot, Keycloak, Keycloak Admin API, REST, Rollen- und Rechteverwaltung, Git

Entwicklung eines Hochschulportals mit zentraler Benutzer- und Rechteverwaltung über Keycloak. Mein Schwerpunkt lag auf der Administration der Keycloak-Umgebung, der Integration der Keycloak Admin API sowie auf der Bereitstellung wiederverwendbarer Spring-Boot-Grundlagen für andere Projektteams.

Aufgaben und Schwerpunkte:

- Administration und Konfiguration der Keycloak-Instanz
- Einrichtung und Pflege von Benutzern, Rollen, Clients und Berechtigungen
- Ansprechpartner im Projektteam für die Keycloak Admin API
- Entwicklung eines eigenen REST-Endpunkts zur Benutzererstellung
- Integration von Authentifizierung und Autorisierung in den Spring-Boot-Service
- Erstellung eines wiederverwendbaren Spring-Boot-Templates für andere Teams
- Verantwortung als Maintainer für das Spring-Boot-Template
- Unterstützung anderer Teams bei der Nutzung und Integration des Templates
- Analyse und Behebung von Problemen im Zusammenhang mit Keycloak und der API

Nutzen:

Durch das Spring-Boot-Template konnten andere Teams auf einer einheitlichen technischen Grundlage aufbauen. Die zentrale Pflege als Maintainer erleichterte zudem die Wiederverwendung und Weiterentwicklung der gemeinsamen Backend-Struktur.